HTX xData Technical Test Question – Engineer

Please follow closely to the Test Instruction

- You can make assumptions to any information not specified within the Test Instruction.
- You can make references to any other technical literature as required.
- Please complete the test and email the required deliverables within 7 calendar days upon receiving this email.
- Reach out to us if you have any queries via email.
- Submit Deliverables to the sender of this test.
- All submitted artifacts must be executable without errors. If a Python environment setup is required, please include the setup scripts in the submission. Failure to comply will result in the submission being considered incomplete.

The purpose of this test is to assess if you have the prerequisite technical ability. You have **a total of 7 calendar days** to complete the list of tasks below.

Complete all the following tasks (1 to 9) by yourself. You will need to document and comment your code properly and state any assumptions made within each task.

Tasks

1. Create a public git repository e.g., https://github.com/fred/myrepo. You may use any of the publicly available repositories like GitHub, Gitlab, etc.

    a) Add a `requirements.txt` for Python libraries
    b) Add a `.gitignore` file
    c) Add a `README.md` file to document the set up and run instructions for your code

2. Clone your repository from Task 1 and create a directory called `asr` in your repository. All code for this task should be kept in this directory. This task will require you to create a hosted microservice to deploy an Automatic Speech Recognition (ASR) AI model that can be used to transcribe any audio files.

    a) AI model to use: `wav2vec2-large-960h` https://huggingface.co/facebook/wav2vec2-large-960h

    This model is developed by Facebook and pretrained and fine-tuned on Librispeech dataset on 16kHz sampled speech audio. Please ensure that your speech input is also sampled at 16kHz. The reference link (above) includes the model card and its usage code.

    b) Write a ping API (i.e. http://localhost:8001/ping via GET) to return a response of "pong" to check if your service is working.

    c) Write an API with the following specifications as a hosted inference API for the model in Task 2a. Name your file `asr_api.py`.

url: http://localhost:8001/asr
Input parameter (content-type: `multipart/form-data`):

- `file` [string type] – the binary of an audio mp3 file

Response (content-type: `application/json`):

- `transcription` [string type] – the transcribed text returned by the model i.e., "*BEFORE HE HAD TIME TO ANSWER A MUCH ENCUMBERED VERA BURST INTO THE ROOM*"
- `duration` [string type] – the duration of the file in seconds i.e., "*20.7*"

Test command using CURL:

```
$ curl -F 'file=@/home/fred/cv-valid-dev/sample-
  000000.mp3' http://localhost:8001/asr
```

d) Data to use: `Common Voice`
Reference: https://www.kaggle.com/datasets/mozillaorg/common-voice

Download and use the following dataset using
https://www.dropbox.com/scl/fi/i9yvfqpf7p8uye5o8k1sj/common_voice.zip?rlkey=lz3dtjuhekc3xw4jnoeogy5yu&dl=0

Write a python file called `cv-decode.py` to call your API in Task 2b to transcribe the 4,076 common-voice mp3 files under cv-valid-dev folder.

Using `cv-valid-dev.csv`, write the generated transcribed text from your API into a new column called `generated_text`. Save this updated file in this folder.

e) Containerise `asr_api.py` using Docker. This will be in `Dockerfile` with the service name `asr-api`. Once the file is successfully processed, your code should delete the file.

3. Create a new directory called `deployment-design` in your repository. As an engineer, you are asked to propose a deployment architecture with the following requirements:

   I. Use elasticsearch container backend (https://www.elastic.co/elasticsearch/) for a search index for all the records within `cs-valid-dev.csv` from Task 2c.
   II. Use search-ui (https://docs.elastic.co/search-ui/overview) as a frontend web application for end users to search on the dataset. This application is deployed as a separate container. The fields that can be searched include `generated_text`, `duration`, `age`, `gender`, and `accent`.

III. Your proposed architecture will be deployed on the public cloud (using either Azure or AWS). You are not allowed to use any managed services.

a) Use draw.io (https://www.drawio.com/) for your proposed architecture. The architecture design should be saved as `design.pdf`.

4. Referring to Task 3a, create a new directory called `elastic-backend` in your repository. All code and scripts for this task should be kept in this directory.

   a) url: http://localhost:9200
   b) index: `cv-transcriptions`
   c) nodes: 2-node cluster
   d) Configuration stored under `docker-compose.yml`
   e) Write a python file called `cv-index.py` to index `cs-valid-dev.csv` into your elasticsearch service.

5. Referring to Task 3a, create a new directory called `search-ui` in your repository. All code and scripts for this task should be kept in this directory.

   a) url: http://localhost:3000/
   b) Configuration stored under `docker-compose.yml`
   c) Configure your search UI to include `generated_text`, `duration`, `age`, `gender`, and `accent`.

6. Deploy your solution to the public cloud as proposed in your design under Task 3a. Use either Azure or AWS free tier.
   a) https://azure.microsoft.com/en-us/free
   b) https://aws.amazon.com/free/

7. Copy your deployment url and include this within `README.md`. Please ensure that this deployment url is accessible publicly.

8. Essay question – propose a model monitoring pipeline and describe how you would track model drift in 500 words. Your answer can be saved as `essay.pdf` under the main repository.

9. Once you have completed this test, submit your git repository url via email. You may undeploy your application once you have received confirmation that your test submission has been received and reviewed.